

TECHNICAL ARCHITECTURE & IMPLEMENTATION REPORT

IP-SAKTI Sahayak

Evidence-Grounded Intellectual Property and Regulatory
Intelligence Copilot for Ayurveda

Problem Statement 045

“Retrieve First. Reason Second. Cite Third. Verify Always.”

Document type: Technical Architecture & Implementation Report

Version: 1.0

Report date: 2 October 2026

Scope: This report documents only PS-045 (IP-SAKTI Sahayak). No content from any other problem statement is included.

Basis: Direct inspection of the current repository source code, database schema, API surface, automated test suite, and running application — no fabricated figures.

Table of Contents

PART I — OVERVIEW

1. Executive Summary	4
2. Problem Statement	4
3. Proposed Solution	4
4. Objectives	5
5. Target Users	5

PART II — ARCHITECTURE

6. Functional Architecture	7
7. High-Level System Architecture	7
8. Technology Stack	8
9. Frontend Architecture	8
10. Backend Architecture	9
11. Database Architecture	10
12. API Architecture	10

PART III — CORE INTELLIGENCE PIPELINE

13. RAG Architecture	11
14. Document Ingestion Pipeline	11
15. Knowledge Sources	12
16. Innovation Profiler	12
17. Patent / Prior-Art Intelligence	13
18. Traditional Knowledge Context	13
19. Scientific Evidence	14
20. Regulatory Intelligence	14
21. Jurisdiction Engine	14
22. Multilingual Architecture	15
23. Citation & Evidence Verification	15
24. Evidence Graph	15
25. Safety & Abstention	15
26. Human-in-the-Loop / Escalation	16

PART IV — PLATFORM

27. Authentication & RBAC	17
28. Security Architecture	17
29. Privacy & Confidentiality	18
30. Admin Architecture	18
31. User Workflow	18
32. Admin Workflow	19
33. AI / LLM Architecture	19
34. AI / RAG Additional Components	19
35. Testing Strategy	19
36. Evaluation Framework	20
37. Performance & Scalability	20
38. Deployment Architecture	20
39. Failure Handling	21

PART V — INVENTORY & TRACEABILITY

40. Complete Feature Inventory 22

41. Additional Implementations Beyond PS-045 22

42. Complete Database Inventory 23

43. Complete API Inventory 24

44. Complete Frontend Inventory 26

45. Complete AI Inventory 26

46. Original PS-045 vs. Current Implementation 26

47. What Has Actually Been Built 27

48. Limitations 27

49. Future Roadmap 27

50. Technical Differentiation 28

APPENDIX

A. Technical Status Legend 29

B. Repository Evidence Index 29

1 Executive Summary

IP-SAKTI Sahayak is a decision-support web application that helps Ayurveda innovators, researchers, startups and IP/regulatory professionals investigate a single question from several angles at once: *what does the available evidence actually say about this innovation's intellectual property position, scientific backing, traditional-knowledge context and regulatory pathway, in a given jurisdiction?* It is built around one architectural discipline, carried through every module: nothing is asserted that was not first retrieved from a cited source, and nothing is shown as supported without a passage-level verification pass.

In plain terms. Imagine typing a question like “can I patent a stress-relief tablet made from Ashwagandha and Turmeric, and what rules apply in India?” Instead of guessing an answer the way a general chatbot would, this system first searches a library of real laws, patents and studies, only writes an answer using what it actually found, double-checks every sentence of that answer against the source it came from, and says “I don’t have enough evidence” rather than making something up when it isn’t sure. It does this separately for India, the USA and Australia, since the rules differ in each. The rest of this report explains, section by section, exactly how much of that is real and working today versus still a work in progress — nothing is overstated.

The current repository is a working full-stack application (a complete product with both the part users see and the part that runs behind the scenes): a Next.js 15 frontend, a FastAPI backend with 108 distinct API routes across 9 routers, a PostgreSQL 16 database (29 tables) with the pgvector extension for dense retrieval, and a hybrid retrieval pipeline that combines vector search, PostgreSQL full-text search, metadata filtering and graph expansion with deterministic reranking. A separate, server-enforced Admin Console sits alongside the user application with its own login, session scope and role check on every request. 69 automated backend tests exercise authentication/RBAC, retrieval correctness, citation verification, jurisdiction isolation, safety abstention, and a full end-to-end user journey.

This report is written to the standard the problem statement demands: every capability is labelled with its real implementation status (**IMPLEMENTED** / **PARTIAL** / **MOCK** / **PLANNED** / **NOT MEASURED**), PS-045’s explicit requirements are kept visibly separate from engineering work added beyond them, and no performance, accuracy or corpus figure appears anywhere in this document unless it was read directly from the running system or the source code during preparation of this report.

Headline honesty note. The corpus behind this system is a curated seed set — **39 documents, 13 sources, 81 chunks** (measured live via GET /api/admin/metrics at report time) — not a production-scale knowledge base, and several demo patents and two guidance notes are explicitly fictional and labelled as such in the product itself. Default embeddings are an offline hashing scheme, not a trained semantic model, unless reconfigured. These facts are expanded in §15, §36 and §48, and are stated here first because they materially bound every other claim in this report.

2 Problem Statement

Ayurveda innovation sits at the intersection of four domains that are each individually complex, and whose combination is where most real decisions actually fail:

- **Intellectual property** — an Ayurvedic formulation’s patentability is shaped by exclusions specific to traditional-knowledge-derived inventions (e.g. India’s Patents Act §3(p), the Biological Diversity Act’s NBA approval requirement), which a generic patent search tool has no concept of.
- **Traditional knowledge** — TK context carries attribution, access and disclosure obligations that differ from ordinary prior art, and restricted repositories (e.g. TKDL) cannot legally or practically be scraped by a side project.
- **Scientific evidence** — a study on an isolated ingredient (e.g. an Ashwagandha extract) does not automatically support a claim about a multi-ingredient finished formulation; conflating the two is a common and consequential error.
- **Regulatory classification** — the same formulation can be a classical ASU medicine, a patent/proprietary ASU medicine, a food, or a health supplement depending on positioning and claims, under *different* frameworks in India (AYUSH/CDSCO/FSSAI), the USA (FDA/DSHEA) and Australia (TGA), which must never be silently blended together.

Overlaying all of this is a language and access problem: much of the relevant literature and regulation is in English, but the people best placed to judge traditional context often work in Hindi; and across all of the above, the cost of a wrong but confident answer is high, while the cost of an honest “insufficient evidence” is low. A generic conversational AI model, prompted with general knowledge, will readily produce fluent, specific-sounding answers to exactly these questions — with no mechanism to say when it does not actually know, no binding between a claim and the passage that is supposed to support it, and no concept of jurisdiction boundaries. That is the gap PS-045 targets: not better chat, but *evidence discipline* applied to a genuinely cross-domain problem.

3 Proposed Solution

IP-SAKTI Sahayak’s answer is to make every module share one evidence pipeline rather than reasoning independently, and to route every user-facing claim through the same retrieve→verify→label discipline regardless of which module produced it.



Figure 3.1 — Core solution flow, as implemented in `app/services/analysis.py`, `regulatory.py`, `patents.py` and `packets.py`.

4 Objectives

Functional Objectives

- Evidence-grounded answers to IP, regulatory, scientific and TK questions
 - Source citations on every substantive claim
 - Structured innovation profiling from a free-text description
 - Patent / prior-art discovery scoped to the innovation's own target markets
 - Scientific evidence discovery that separates ingredient- from formulation-level support
- Jurisdiction-specific regulatory classification and pathway guidance
 - Multilingual interaction (English / Hindi)
 - Explicit citation verification, not just citation display
 - Safe abstention when evidence is missing, conflicting or out of scope
 - Human escalation with a complete evidence packet
 - End-to-end auditability of user and admin actions

Technical Objectives

- Hybrid retrieval (dense + lexical + metadata + graph) over a relational store, avoiding a second database system
 - Deterministic, explainable reranking (not an opaque learned ranker)
 - A pluggable LLM provider layer that degrades to extractive-only output rather than failing when no key is configured
- Hard server-side enforcement of workspace isolation and role checks, not just UI hiding
 - A fully separate admin session scope, re-checked on every request
 - Secure handling of confidential innovation data (hashing, validation, upload sanitisation)
 - A real automated test suite covering security-critical paths, not just happy paths

5 Target Users

INTENDED USER (PS-045 DOMAIN)	IMPLEMENTED APPLICATION ROLE	WHAT THEY CAN ACTUALLY DO TODAY
Ayurveda innovator / startup founder	USER role, workspace role RESEARCHER or OWNER	Create innovations, run research, view evidence/patents/regulatory, generate reports, raise escalations
Institutional researcher	USER, workspace role RESEARCHER/VIEWER	Same as above, scoped to workspace membership
IP / regulatory reviewer	USER, workspace role REVIEWER	Review patent-match status, resolve escalations and feedback within a workspace

INTENDED USER (PS-045 DOMAIN)	IMPLEMENTED APPLICATION ROLE	WHAT THEY CAN ACTUALLY DO TODAY
Platform administrator	ADMIN global role and a separate admin-console session	Manage users/workspaces/sources/documents, monitor RAG & citation quality, run evaluations, review audit logs, cross-workspace escalations/feedback

There is no distinct implemented role for “Ayurveda domain expert” beyond the workspace REVIEWER role — domain-expert review is a workflow (escalation + reviewer assignment), not a separate account type.

6 Functional Architecture

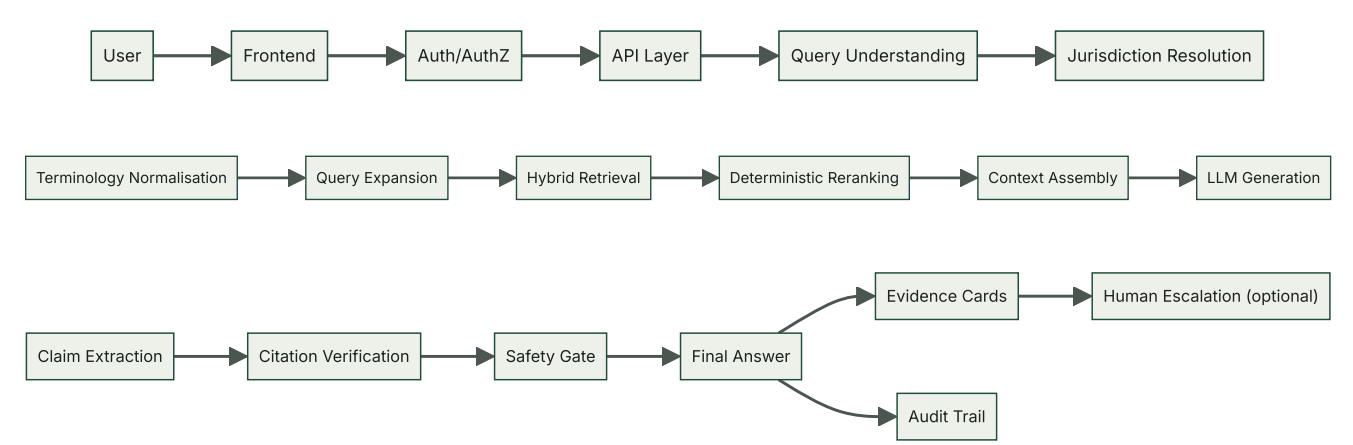


Figure 6.1 — Functional pipeline (read top to bottom across the three rows). All stages shown are **IMPLEMENTED** and traceable to `app/services/research.py`, `query_understanding.py`, `retrieval.py`, `verification.py`, `llm.py`.

STAGE	STATUS	EVIDENCE
Intent / language detection, jurisdiction resolution	IMPLEMENTED	services/query_understanding.py
Terminology normalisation & expansion	IMPLEMENTED	services/terminology.py, models/orm.py::Term
Hybrid retrieval + deterministic rerank	IMPLEMENTED	services/retrieval.py
LLM generation	PARTIAL — depends on configured provider; extractive-only if none	services/llm.py
Claim extraction & citation verification	IMPLEMENTED	services/verification.py
Safety / abstention gate	IMPLEMENTED	services/research.py
Human escalation	IMPLEMENTED	services/packets.py, api/endpoints/review.py

7 High-Level System Architecture

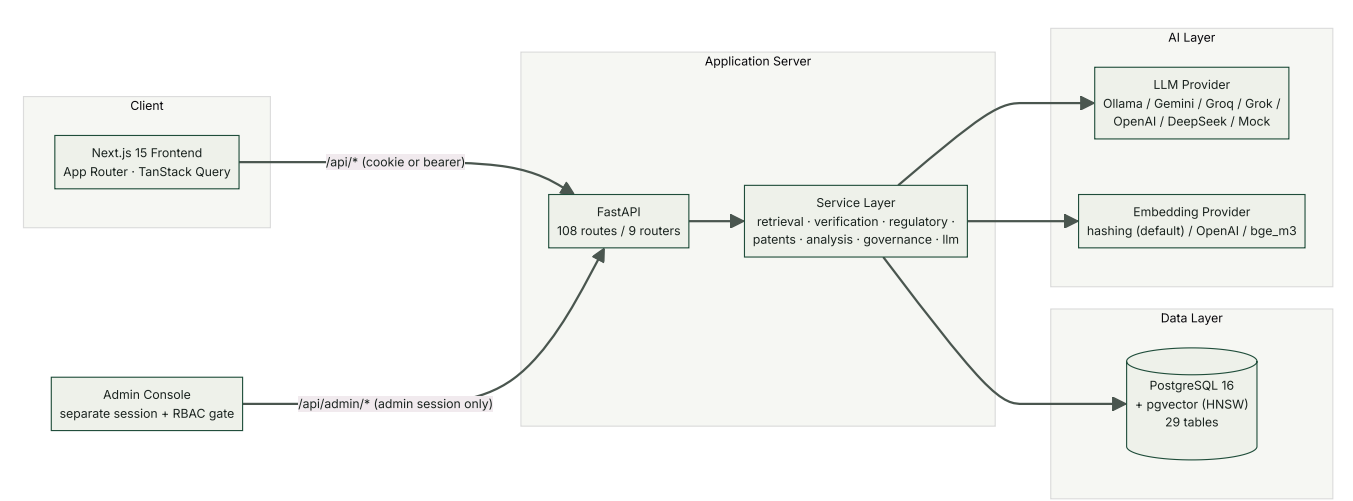


Figure 7.1 — High-level system architecture. Every box is present in the running repository; no component shown is aspirational.

There is a single application database (PostgreSQL with the pgvector extension) — there is no separate vector database product; dense retrieval runs as an HNSW-indexed similarity query inside the same Postgres instance that holds relational data. There is no message queue, no cache layer (e.g. Redis), and no separate search engine (e.g. Elasticsearch) anywhere in the stack.

8 Technology Stack

LAYER	TECHNOLOGY	PURPOSE	STATUS
Frontend framework	Next.js 15.1.6 (App Router), React 19	UI, routing, server-side rendering of static pages	IMPLEMENTED
Frontend data layer	TanStack Query 5.104	Server-state fetching/caching	IMPLEMENTED
Frontend styling	Tailwind CSS 3.4.17	Utility-first styling	IMPLEMENTED
Evidence graph visualisation	@xyflow/react 12.12 (React Flow)	Interactive node/edge graph	IMPLEMENTED
Backend framework	FastAPI (>=0.110, installed 0.142.1)	API server	IMPLEMENTED
ORM / migrations	SQLAlchemy 2.x (installed 2.1.1), Alembic (installed 1.20.0, 4 migrations)	Data modelling & schema evolution	IMPLEMENTED
Database	PostgreSQL 16 (pgvector/pgvector:pg16 image)	Relational + vector storage	IMPLEMENTED
Vector search	pgvector HNSW index (cosine)	Dense semantic retrieval	IMPLEMENTED
Full-text search	PostgreSQL tsvector + GIN index	Lexical/BM25-style retrieval	IMPLEMENTED
LLM generation	Provider abstraction: Ollama (local default in this deployment), Gemini, Groq, Grok, OpenAI, DeepSeek, Mock fallback	Answer synthesis	PARTIAL — provider-dependent
Embeddings	Hashing n-gram (default), OpenAI text-embedding-3-small, bge_m3 (alternatives)	Dense vector generation	PARTIAL — default is not a trained semantic model
Authentication	bcrypt password hashing, PyJWT, httpOnly cookies	Session management	IMPLEMENTED
OCR	Tesseract via pytesseract	Scanned-document text extraction	IMPLEMENTED
PDF parsing	PyMuPDF	Native-text PDF extraction	IMPLEMENTED
Containerisation	Docker Compose, 2 Dockerfiles (backend, frontend)	Local/deployment packaging	IMPLEMENTED
CI/CD	—	—	NOT IMPLEMENTED — no .github/workflows or equivalent pipeline found

9 Frontend Architecture

The frontend is a Next.js 15 App Router application with three route trees: public pages, the authenticated user application (/app/**), and a fully separate authenticated admin console (/admin/**) that never shares session state with the user app (two distinct React context providers — lib/providers.tsx's useApp() and lib/admin.tsx's useAdmin() — and two distinct session cookies). State management is deliberately minimal: TanStack Query for server state, React Context for session/language/workspace/jurisdiction preference, and localStorage for client-only persistence — no Redux/Zustand.

Page inventory (55 distinct routes)

AREA	ROUTES	COUNT
Public	/, /login, /register, /admin/login	4
User app — core	/app (dashboard), /app/innovations, /app/innovations/new, /app/assistant, /app/documents(+[did]), /app/sources, /app/coverage, /app/escalations(+[eid]), /app/reports(+[rid]), /app/feedback, /app/audit, /app/settings, /app/privacy	16
User app — per-innovation tabs	/app/innovations/[id] (summary) + /profile, /classification, /scientific, /patents, /tk, /regulatory, /evidence, /graph, /gaps, /review	11

AREA	ROUTES	COUNT
User app — choosers	/app/analysis/[view] (4 views), /app/research/[kind] (4 kinds)	8
Admin console	/admin, /admin/dashboard, /users, /workspaces, /documents, /sources, /citations, /rag, /evaluation, /escalations(+[id]), /feedback, /audit-logs, /settings	14

Shared component library

components/ui.tsx provides the design-system primitives (Button, Card, Badge family — including JurisdictionBadge, SupportBadge, SeverityBadge, TierBadge, FreshnessBadge, DemoBadge — Spinner, ProgressSteps, EmptyState, ErrorState, PageHeader, Stat, Tabs, Drawer, Field, Notice, Popover, ProfileMenu). components/evidence.tsx provides domain cards (EvidenceCard, PatentCard, StudyCard, RegulatoryCard, SourceDrawer, SearchTrace). Error states, loading states and empty states are implemented as reusable components and used consistently, not ad hoc per page.

10 Backend Architecture

The backend follows a conventional layered FastAPI structure: api/endpoints/*.py (HTTP routing + request/response shaping only), services/*.py (business logic — retrieval, verification, regulatory classification, patent matching, analysis/gaps, governance, ingestion, evaluation, LLM access, audit), models/orm.py (SQLAlchemy models), schemas.py (Pydantic request/response contracts), core/ (config, security, response envelope, error handling).

ROUTER FILE	ROUTES	RESPONSIBILITY
auth.py	10	Registration, user login/session, admin login/session (separate scope), password change
workspaces.py	5	Workspace CRUD, membership
innovations.py	24	Innovation CRUD, profiling, analysis, evidence, scientific, TK, patents, gaps, risk, graph, classification, regulatory, summary, provisions
chat.py	6	AI Research Assistant conversations
search.py	8	Standalone search (general/patents/scientific/regulatory), citation verify, classification, regulatory overview
review.py	12	Escalations, reports, audit, feedback
corpus.py	9	Sources, documents, uploads, ingestion jobs, coverage, privacy
dashboard.py	1	Workspace dashboard metrics
admin_console.py	28	Everything under /api/admin/*, gated by a router-level require_admin dependency
health.py	4 (mounted twice)	Liveness/readiness for overall, database, vector index, LLM

Error handling is centralised: a single AppError exception type carries an HTTP status, a machine-readable code and a message, and core/responses.py installs handlers so every response — success or failure — uses one envelope shape ({success, data, meta} or {success:false, error:{code,message}}), with stack traces never returned to the client. Structured JSON logging is installed globally and explicitly never logs request bodies (innovation text may be confidential).

11 Database Architecture

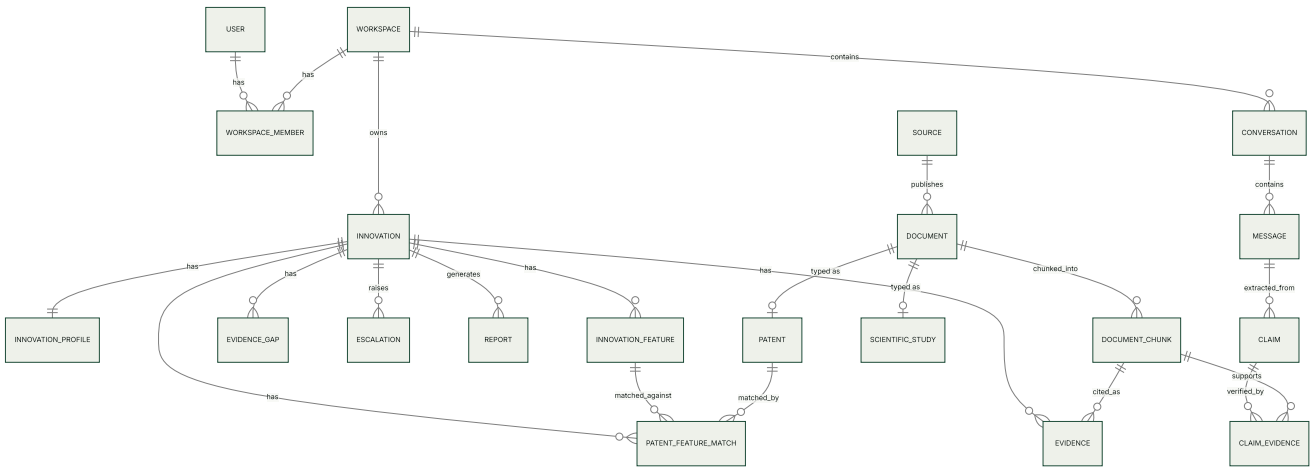


Figure 11.1 — Conceptual entity-relationship model (simplified to the 17 most structurally significant of the 29 tables; see §42 for the complete inventory).

All 29 tables live in one PostgreSQL schema. Enumerated status fields (UserRole, WorkspaceRole, Confidentiality, InnovationStatus, FeatureType, SupportStatus, Severity, EscalationStatus, EscalationType) are stored as length-bound VARCHAR columns with application-level enum validation (SQLAlchemy native_enum=False), not native Postgres enum types — a deliberate simplicity trade-off that keeps Alembic migrations simpler at the cost of no database-level constraint on those values. DocumentChunk.embedding is a pgvector column with an HNSW index (cosine distance, m=16, ef_construction=64); DocumentChunk.search_vector is a generated tsvector column with a GIN index, enabling the hybrid dense+lexical retrieval described in §13.

12 API Architecture

108 distinct route definitions (112 registrations counting the health router’s dual mount) are exposed under a single /api prefix (health checks are also mounted unprefixed). Every response uses the same JSON envelope. Cookie-authenticated state-changing requests must carry X-Requested-With: ipsakti as a CSRF defence; API clients may instead use Authorization: Bearer. See §43 for the complete endpoint table (all 108 routes, grouped by router, with method/purpose/auth).

AUTH TIER	MECHANISM	APPLIES TO
None	—	Registration, login endpoints, health checks, session-state probes (which never 401)
User session	Scoped JWT (scope=user) in httpOnly cookie or Bearer token	All /api/* app routes
Workspace role	User session + ctx.require(role) minimum-role check (VIEWER<REVIEWER<RESEARCHER<ADMIN<OWNER)	Write operations within a workspace
Admin session	Separate scoped JWT (scope=admin) + live role=ADMIN re-check every request	All /api/admin/* routes (router-level dependency)

13 RAG Architecture

RAG stands for Retrieval-Augmented Generation — a search step that finds real passages first, followed by a generation step that is only allowed to write using those passages, instead of an AI model answering from memory alone.

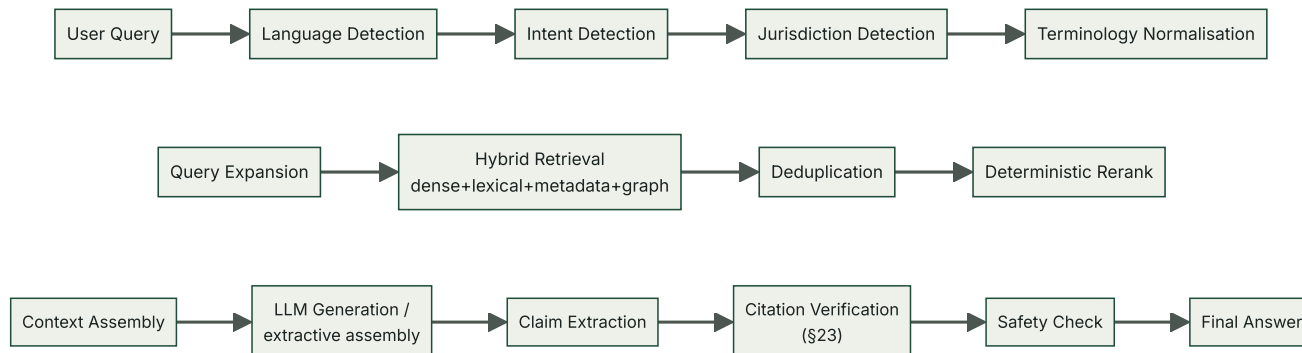


Figure 13.1 — RAG pipeline (read top to bottom across the three rows), as implemented across `services/query_understanding.py`, `terminology.py`, `retrieval.py`, `research.py`, `verification.py`.

STEP	INPUT	OUTPUT	TECHNOLOGY	STATUS
Language / intent detection	Raw query text	Language tag, intent class (GENERAL/INNOVATION/PATENT/SCIENTIFIC/REGULATORY/CROSS_JURISDICTION)	Rule-based heuristics in <code>query_understanding.py</code>	IMPLEMENTED
Jurisdiction detection	Query text + explicit selection	Jurisdiction list, or a clarification/refusal signal	<code>detect_jurisdictions()</code>	IMPLEMENTED
Terminology normalisation	Query + expanded terms	Canonical term, flagged ambiguity (e.g. "Brahmi")	Term table + <code>TerminologyEngine</code>	IMPLEMENTED
Hybrid retrieval	Expanded query + filters	Ranked chunk candidates	pgvector HNSW + GIN tsvector + SQL metadata filter + graph expansion, in one SQL layer (<code>retrieval.py</code>)	IMPLEMENTED
Reranking	Candidate set	Final ordered set + <code>why_retrieved</code> explanation per item	Deterministic weighted scoring (no cross-encoder/ML reranker)	IMPLEMENTED
Generation	Assembled context	Draft answer	Configured LLM provider, or direct passage quotation if none configured	PARTIAL — provider-dependent
Claim extraction + citation verification	Draft answer	Per-claim support verdict	<code>verification.py</code> , see §23	IMPLEMENTED
Safety / abstention	Verified answer + evidence state	Final answer or an explicit abstention/clarification	<code>research.py</code> safety gates	IMPLEMENTED

For cross-jurisdiction queries, the pipeline does not merge sources into one generic retrieval call: it runs one full retrieval per selected jurisdiction and keeps results structurally separate before composing the final answer, specifically to prevent one jurisdiction's sources crowding out another's (`research.py`'s `JURISDICTION_COMPARISON` branch).

14 Document Ingestion Pipeline

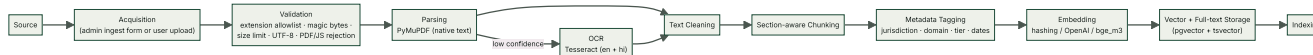


Figure 14.1 — Ingestion pipeline, implemented in `services/ingestion.py` and the chunker shared with the earlier prototype under `rag//ingestion/`.

Every ingestion run is tracked as an IngestionJob row with a per-step status array, visible to admins. Duplicate detection is by content hash. Instruction-like text inside an ingested document (a prompt-injection attempt embedded in a source file) is flagged at ingestion time, down-ranked in retrieval, and excluded from key-point synthesis — this is tested explicitly (tests/test_retrieval.py). Jurisdiction on upload is validated against a closed set (IN/US/AU/INTERNATIONAL) rather than accepted as a free string.

15 Knowledge Sources

As measured live from the running database at report time (GET /api/admin/metrics, GET /api/coverage): **13 sources, 39 documents, 81 chunks**, of which **30 documents are labelled real/seed-summary** and **9 are explicitly labelled fictional demo content**.

CATEGORY	EXAMPLES IN THE CORPUS	LABEL	STATUS
Primary / official	The Patents Act 1970 (India) excerpts, Biological Diversity Act, AYUSH/CDSCO/FSSAI rules, US FDA/DSHEA provisions, Australia TGA requirements	SEED_SUMMARY — short excerpts/close summaries of real instruments	PARTIAL — summarised, not the full official text; verification against the Gazette/regulator text is explicitly recommended to the user in-product
Authoritative secondary	Peer-reviewed study abstracts on Ashwagandha, Turmeric/curcumin, Triphala	SEED_SUMMARY	PARTIAL — a small, hand-curated set, not a literature-database integration
Supporting / demonstration	7 fictional patents (DEMO-... numbers, 3 IN / 2 US / 1 AU / 1 EP), 2 conflicting fictional guidance notes	DEMO_FICTIONAL	MOCK — explicitly fictional, labelled as such everywhere they appear
User-provided	Workspace uploads	PENDING_REVIEW	IMPLEMENTED workflow; content itself is unverified by definition

There is no live/production source integration (no scheduled crawler, no external API pull from a patent office, regulator or journal database) — every document currently in the corpus was added via the seed script or a manual ingestion call. **The system has no access to TKDL (Traditional Knowledge Digital Library) and does not scrape or reproduce restricted TK records** — this is a deliberate boundary, not a missing integration.

16 Innovation Profiler

A 7-step wizard (frontend/app/app/innovations/new/page.tsx) collects: basic info (name, description, intended use), ingredients (common/Sanskrit/Hindi/botanical/chemical name, extract, quantity, source), formulation, process (extraction/manufacturing), claims, target markets (multi-select IN/US/AU — the authoritative field, stored as Innovation.target_markets), and confidentiality level. Submission calls POST /api/innovations, which builds a structured InnovationProfile and a set of typed InnovationFeature rows (ingredient/composition/process/extraction/delivery/dosage-form/claim/use/manufacturing/other).

Profile generation explicitly flags rather than silently resolves ambiguity — e.g. a common name like “Brahmi” maps to two distinct botanical species (*Bacopa monnieri* or *Centella asiatica*) and is surfaced to the user as an open question, not auto-picked. A disease/treatment claim in free text is also flagged, since it changes the regulatory analysis materially. The profile and its target markets are what every downstream module (patents, regulatory, gaps) reads — there is no second, independent “jurisdiction” or “markets” field elsewhere that could silently diverge. Status: **IMPLEMENTED**, end-to-end tested (tests/test_integration_flow.py).

17 Patent / Prior-Art Intelligence

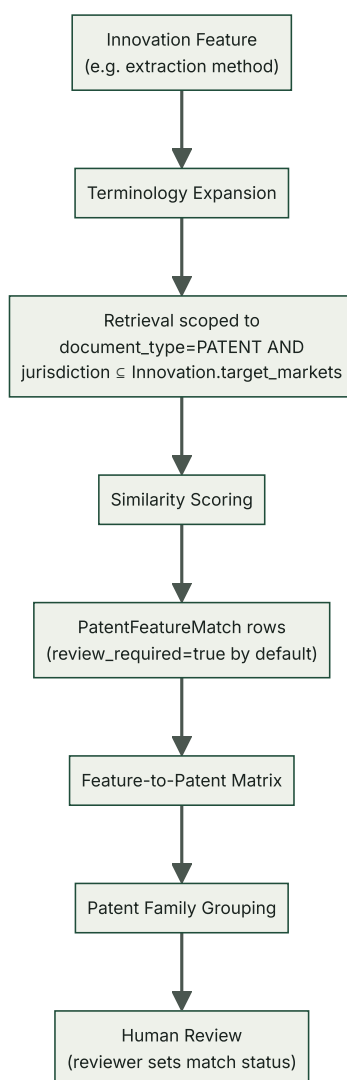


Figure 17.1 — Patent/prior-art matching flow (*services/patents.py*). **Every match is explicitly flagged for human review and never presented as a patentability or infringement verdict.**

Two code paths exist: `match_innovation()` (builds the innovation's own feature-to-patent matrix, persisted as `PatentFeatureMatch` rows) and `search_patents()` (ad-hoc keyword/concept search over the patent corpus, used by the standalone research page). Both now filter by jurisdiction at the SQL layer rather than discarding results after retrieval. A seed-data correctness detail worth recording here: one fictional patent is tagged jurisdiction EP, which is outside the three jurisdictions this system formally supports; it remains discoverable via general search but is structurally excluded from innovation-scoped matching, since `target_markets` can only ever contain IN/US/AU.

Patent coverage in the seed corpus is thin and uneven — 3 India, 2 USA, 1 Australia, 1 EP, all fictional — which materially limits how much real prior-art signal this module can currently produce; this is stated plainly rather than implied away. Status: **IMPLEMENTED** (mechanism), operating over **MOCK** data (corpus content).

18 Traditional Knowledge Context

TK-related documents in the corpus (ABS/disclosure law excerpts, TK context notes) are retrieved through the same hybrid pipeline, scoped to the TK_ABS domain. Access level on each TK-tagged source is explicit (`PUBLIC` / `RESTRICTED_RECORDS_PUBLIC_INFO` / `WORKSPACE` / `RESTRICTED`); documents marked `RESTRICTED` are excluded from retrieval entirely at the SQL layer (`d.access_level <> 'RESTRICTED'` is a hard clause in every retrieval query). A dedicated evidence gap (`nba-approval`) is raised whenever an innovation targets India or sources an ingredient from India, pointing at the Biological Diversity Act's NBA approval requirement. Status: **IMPLEMENTED** for the mechanism; **MOCK** /limited for underlying TK content, since there is no restricted-database integration by design.

19 Scientific Evidence

Scientific retrieval is intentionally jurisdiction-agnostic (science doesn't belong to a country) but is not ingredient/formulation-agnostic: every `ScientificStudy` record carries an `evidence_level` of `INGREDIENT` or `FORMULATION`, and the scientific-evidence view explicitly states, for every ingredient-level card, that it *does not establish* efficacy of the finished multi-ingredient formulation — this distinction is enforced in code (`services/analysis.py`) and asserted in the test suite, not left to prose. The seed corpus contains 6 scientific studies, all ingredient-level (Ashwagandha, Turmeric/curcumin, Triphala) — there is currently no finished-formulation-level study in the corpus, which the gap-detection logic surfaces as a formulation-evidence gap by default. Status: **IMPLEMENTED** mechanism, **MOCK** /limited underlying data.

20 Regulatory Intelligence

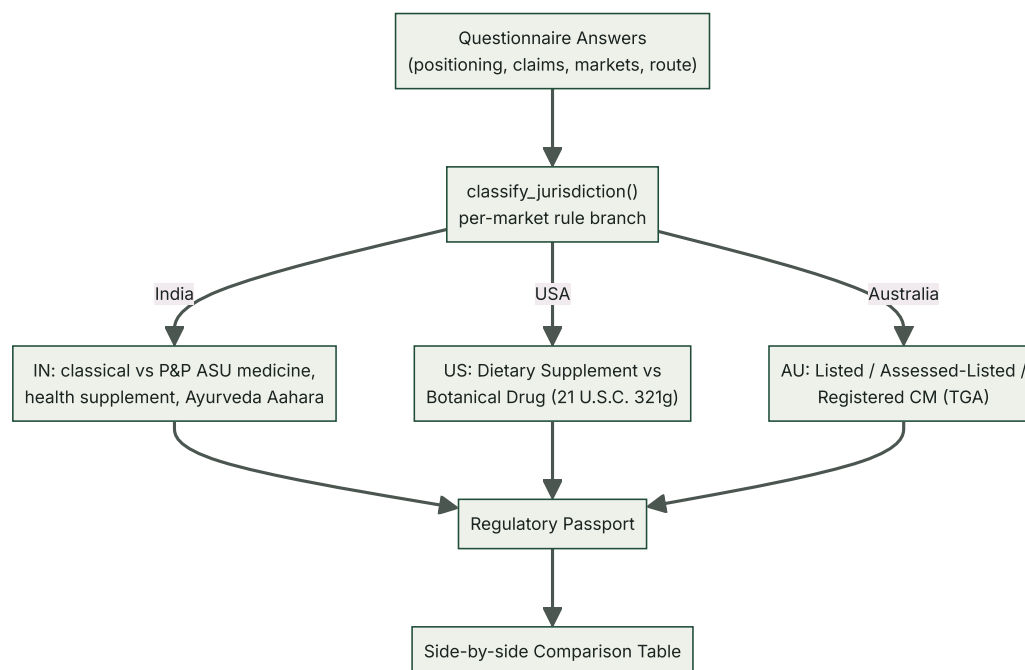


Figure 20.1 — Regulatory classification flow (`services/regulatory.py`). Each jurisdiction has materially different branch logic, not a shared template with a label swap.

For every candidate classification, the system records a confidence score, the assumptions it made, the facts that could change the answer, and any missing questions — explicitly never a final determination. `comparison()` always shows all three jurisdictions side by side regardless of the innovation's own target markets (an intentional design choice for exploratory comparison), while the *passport* view is scoped to the innovation's actual targets. Status: **IMPLEMENTED**.

21 Jurisdiction Engine

Three jurisdictions are supported: **India, USA, Australia**. Selection happens on the landing page (a multi-select, persisted via a shared client-side preference mirroring the existing workspace-selection pattern) and is carried automatically through registration/login into the dashboard, chat assistant and research forms — not re-asked on every page. At the innovation level, `Innovation.target_markets` is the authoritative per-innovation override, feeding regulatory classification, evidence-gap detection, and (as of this report) patent matching.

Jurisdiction is enforced, not cosmetic, at the retrieval layer: a hard SQL filter restricts results to the selected jurisdiction(s) plus international instruments, and a reranking term boosts jurisdiction-matching results further. A dedicated `JURISDICTION_COMPARISON` intent runs fully separate retrieval per jurisdiction rather than one blended query. If a user asks about a jurisdiction other than the one currently selected, the system detects the mismatch and asks for clarification rather than silently answering with the wrong jurisdiction's rules. If evidence for the selected jurisdiction is insufficient, the system says so explicitly rather than filling the gap from another jurisdiction. Admin analytics (RAG and citation monitoring) break activity down by jurisdiction. Status: **IMPLEMENTED**, with 9 dedicated backend tests covering filter correctness, gap parity (India/USA/Australia), and the admin analytics rollups.

22 Multilingual Architecture

Supported UI languages: **English and Hindi** — matching the PS-045 MVP target exactly; no other language is implemented. Hindi queries are handled primarily via terminology expansion over the English-language corpus (Sanskrit/Hindi/botanical/chemical name mapping), not via a parallel Hindi corpus or Hindi-trained embeddings. When an LLM is configured, Hindi questions can additionally be answered in Hindi and Hindi queries can be translated for retrieval — with an explicit guard that rejects an LLM “translation” that is implausibly long or that invents a named legal instrument not present in the original query (added specifically after an observed hallucination of a fictitious “Traditional Knowledge Digital Library Act, 2005”). OCR supports Hindi text extraction when the Tesseract Hindi language pack is installed. Status: **IMPLEMENTED** for EN/Hi UI and terminology-based retrieval; **PARTIAL** for LLM-based Hindi generation/translation (provider-dependent, with a demonstrated past failure mode now guarded against rather than eliminated by design).

23 Citation & Evidence Verification

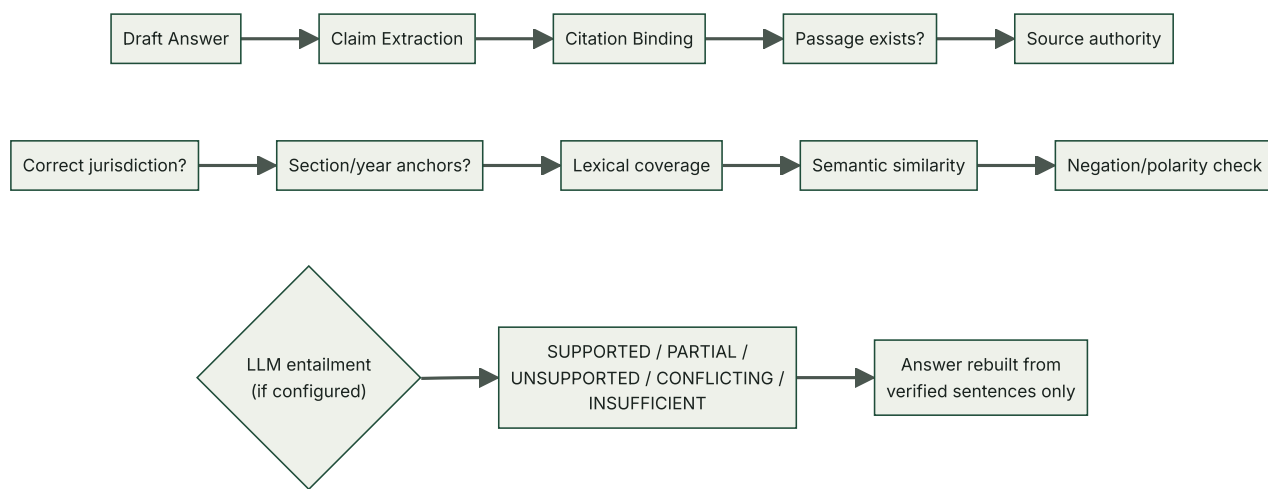


Figure 23.1 — Citation verification pipeline (read top to bottom across the three rows; `services/verification.py`). This is one of the most heavily iterated subsystems in the project; see §39 for specific failure modes it was built to close.

This is deliberately not a cosmetic “show a source link” feature: the system reconstructs its final displayed answer only from sentences that survive verification against the retrieved passages, and drops/reports the rest in a `limitations` field rather than showing unverified prose. An LLM-reported verdict string is validated against an allowed set before being trusted — a malformed or out-of-vocabulary value from the model is ignored rather than acted on. Negation/polarity checking compares only against the single best-matching sentence (an earlier version produced false `CONFLICTING` verdicts from negation words occurring anywhere in a passage; this was fixed and is covered by a regression test). Status: **IMPLEMENTED**, with dedicated tests for supported/contradicted/unsupported/conflicting cases and admin-side monitoring (§30).

24 Evidence Graph

Per innovation, a graph is constructed (`services/analysis.py::evidence_graph()`) with first-class node types for Innovation, Feature, Evidence, Regulation, and **Jurisdiction** (jurisdiction is a real node type, not a label on an edge), with provenance retained on every edge back to the originating passage. The frontend renders this as an interactive node/edge diagram using React Flow (@xyflow/react) on the `/app/innovations/[id]/graph` tab. Status: **IMPLEMENTED**.

25 Safety & Abstention

The governing principle, enforced in code rather than only stated in a prompt: **do not guess when evidence is insufficient**. Concretely implemented gates:

- Insufficient evidence → explicit `INSUFFICIENT` status with what was searched and what is missing
- Conflicting sources → shown side by side, never silently merged
- Jurisdiction not selected / detected → clarification request, not a guess
- Jurisdiction outside IN/US/AU → explicit refusal (e.g. EU questions)
- Outdated/superseded sources → freshness-labelled (Current / Recently checked / Potentially stale / Superseded / Unknown), excluded unless explicitly requested
- Unsupported claims → dropped from the final answer, not merely flagged

- Medical/legal-certainty questions → explicit boundary refusal (this is not a medical or legal determination)
- Restricted-TKDL-style requests → explicit refusal, not a best-effort answer
- Prompt injection in the query or in retrieved documents → detected and never followed

Status: **IMPLEMENTED**, each gate covered by at least one dedicated test in tests/test_rag.py.

26 Human-in-the-Loop / Escalation

Four escalation types exist (IP_REVIEW, REGULATORY_REVIEW, DOMAIN_REVIEW, HUMAN_REVIEW), each created with a full evidence packet (profile, findings, gaps, citations, open questions, audit trail) attached, not just a note. Escalations carry a status workflow (OPEN→IN_REVIEW→RESOLVED→CLOSED) and, in the admin console, can be assigned to a named reviewer with accumulating reviewer notes. Evidence gaps above a severity threshold are a direct trigger surfaced to the user for escalation, not merely logged. Status:

IMPLEMENTED, end-to-end tested.

27 Authentication & RBAC

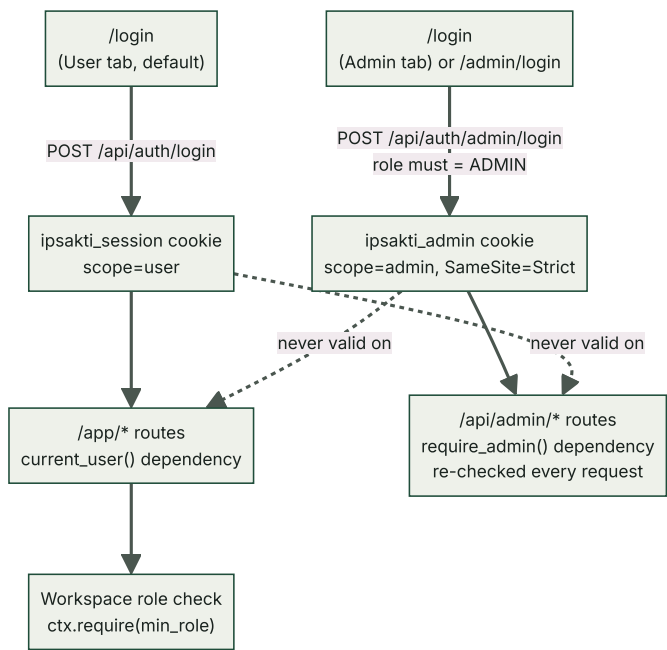


Figure 27.1 — Authentication & RBAC. The two session scopes are structurally incompatible — a user-scoped token is rejected by every admin route, and vice versa, verified by a dedicated access-rules-matrix test.

Two independent RBAC layers exist: a global `User.role` (ADMIN/USER) that only gates the separate admin console, and a per-workspace `WorkspaceRole` (OWNER>ADMIN>RESEARCHER>REVIEWER>VIEWER) that gates actions inside the app — an ADMIN-role user gets no special workspace privileges merely by being a global admin; they must also hold a workspace role, same as anyone else. Passwords are bcrypt-hashed; sessions are JWTs with a scope claim validated on every decode; admin sessions use a stricter cookie (SameSite=Strict, shorter TTL) and additionally require a live re-check of `role=ADMIN` on every request, not only at login (so a demotion takes effect immediately, not at next login). Login attempts are throttled with a sliding-window limiter. Status: **IMPLEMENTED**, with 15 dedicated security tests covering the full access-rules matrix (unauthenticated→redirect, USER→admin routes forbidden, ADMIN→admin routes allowed, cross-user data isolation even from the admin document view).

28 Security Architecture

CONTROL	IMPLEMENTATION	STATUS
Password hashing	bcrypt, with enforced password rules on registration (min length, letter + number)	IMPLEMENTED
Session transport	httpOnly, SameSite cookies (Strict for admin, Lax for app); Bearer token for API clients	IMPLEMENTED
CSRF defence	X-Requested-With: ipsakti required on cookie-authenticated writes	IMPLEMENTED
Input validation	Pydantic schemas on every request body; SQLAlchemy bound parameters only (no raw string SQL)	IMPLEMENTED
Output escaping	React's default escaping; no raw HTML rendering of untrusted content	IMPLEMENTED
Upload validation	Extension allowlist, magic-byte signature check, size limit, UTF-8 validation, rejection of PDFs carrying JavaScript/Launch actions, duplicate-hash detection	IMPLEMENTED
Prompt-injection defence	Instruction-like text in queries and ingested documents is detected, down-ranked, labelled, excluded from key points, and the generation prompt explicitly forbids following it	IMPLEMENTED
Rate limiting	Per-user, per-minute limits on chat/search/ingest/report/login endpoints	PARTIAL — in-process only, not distributed; would not hold a limit correctly across multiple server instances
Security headers	X-Content-Type-Options, X-Frame-Options: DENY, Referrer-Policy, Content-Security-Policy (non-docs routes), Cache-Control: no-store by default	IMPLEMENTED

CONTROL	IMPLEMENTATION	STATUS
CORS	Restricted to configured origins (not wildcard)	IMPLEMENTED
Secrets handling	Loaded from .env, never committed; admin settings view masks secret values	IMPLEMENTED
Audit logging	Append-only audit_logs table; requester IP stored only as a salted hash, never raw	IMPLEMENTED
Dependency / SAST scanning, secret scanning in CI	—	NOT IMPLEMENTED — no CI pipeline exists to run one
Penetration testing / third-party security audit	—	NOT MEASURED — none performed; no security certification of any kind is held or claimed

29 Privacy & Confidentiality

Workspaces can be marked confidential; questions originating from a confidential workspace are redacted in admin-facing monitoring views (RAG monitoring, citation monitoring) rather than shown in full. Innovation data, conversations, uploaded documents and search history are scoped to the owning workspace and are not visible across workspaces except through the admin console (itself gated by a separate session, §27). A dedicated /api/privacy endpoint and /app/privacy page state the retention policy, what data is sent to the configured LLM provider, and the upload notice shown before any file leaves the browser. Retention purge is implemented as an admin-triggered (and dry-run-able) job, not a scheduled background process. Status: **IMPLEMENTED** for the mechanisms described; **NOT MEASURED** for any formal data-protection compliance certification (none is claimed).

30 Admin Architecture

AREA	CAPABILITY	STATUS
Users	Search, activate/deactivate, change role (self-lockout and last-admin-standing guards in place)	IMPLEMENTED
Workspaces	List all workspaces with member/innovation counts	IMPLEMENTED
Source registry	Register/edit/disable sources, set authority tier and jurisdiction	IMPLEMENTED
Documents	Cross-workspace search, ingest, review (mark checked/approve/reject/restore/mark superseded), re-index	IMPLEMENTED
RAG monitoring	Query counts, abstention/failed-retrieval rates, latency percentiles, breakdowns by type/status/mode/intent/ jurisdiction , recent system errors	IMPLEMENTED
Citation monitoring	Support-status/claim-type/ jurisdiction breakdowns, recent verification failures with entailment scores	IMPLEMENTED
Evaluation	Runs the seeded evaluation question set against the live pipeline on demand; shows the latest run and history	IMPLEMENTED mechanism; see §36 for what it does <i>not</i> measure
Escalations	Cross-workspace list, reviewer assignment, status/notes	IMPLEMENTED
Feedback	Cross-workspace reported-issue queue with answer snapshot	IMPLEMENTED
Audit logs	Filterable by category/action/user/entity/date; CSV export	IMPLEMENTED
Settings	Read-only effective configuration view, secrets masked	IMPLEMENTED
Retention	Policy-driven purge, dry-run supported	IMPLEMENTED

31 User Workflow

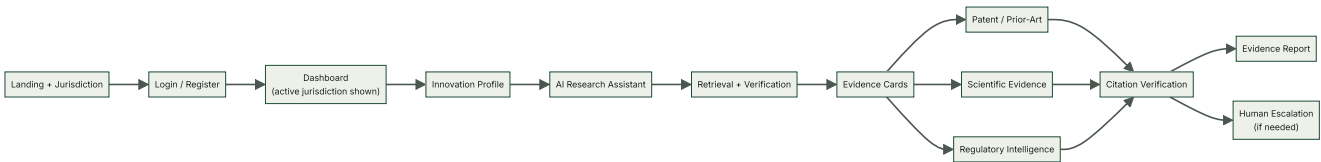


Figure 31.1 — Implemented end-to-end user journey. Every step shown is reachable in the running application with no placeholder screens.

32 Admin Workflow

Admin login (separate URL/session) → Dashboard (system health, corpus size, recent errors)

→ Source Registry (register/curate sources)

→ Documents (ingest → review → approve/reject → re-index)

→ RAG / Citation Monitoring (spot failure patterns, by jurisdiction)

→ Evaluation (run the gold question set, inspect results)

→ Escalations (assign reviewers, track resolution)

→ Feedback (triage reported issues)

→ Audit Logs (investigate an action, export for records)

33 AI / LLM Architecture

An LLMProvider abstract base class is implemented once (OpenAICompatibleProvider) and parameterised per provider: ollama (local, free, private — the default in this deployment's .env), gemini, groq, grok, openai, deepseek, plus a mock fallback (the default in the committed .env.example, i.e. a fresh checkout with no key configured runs in extractive-only mode by default). There is no agent framework, no tool-calling/function-calling orchestration, and no multi-agent system — the LLM is called directly for two purposes: answer generation (constrained to the assembled retrieved context) and, when configured, a capped claim-entailment check (at most 3 LLM calls per answer, for cost/latency control). System prompts explicitly instruct the model to answer only from the provided context, to never follow instruction-like text found inside retrieved content, and to mark uncertainty rather than invent specifics. If the configured provider fails at runtime, the system falls back to extractive mode automatically and states so in the answer rather than erroring out.

CAPABILITY	STATUS
Multi-provider abstraction with automatic extractive fallback	IMPLEMENTED
Configurable temperature/model per provider	IMPLEMENTED (via environment configuration)
Structured-output / JSON-mode enforcement from the LLM	PARTIAL — the system does not rely on provider-native structured output; it parses/validates free text and discards anything that doesn't fit an allowed verdict set
Tool/function calling, autonomous agents	NOT IMPLEMENTED
Model fine-tuning	NOT IMPLEMENTED — only off-the-shelf provider models are used

34 AI / RAG Additional Components

COMPONENT	CLASSIFICATION	STATUS
Query expansion via terminology engine	Required/Supporting for PS-045 retrieval quality	IMPLEMENTED
Deterministic (non-ML) reranking	Supporting	IMPLEMENTED
Citation/entailment verification	Required for PS-045's evidence-grounding goal	IMPLEMENTED
Evidence/gap scoring, risk-map aggregation	Additional	IMPLEMENTED
Hallucinated-translation guard (rejects implausible/invented legal-instrument translations)	Additional, added after an observed failure	IMPLEMENTED
Evidence-report generation (structured + Markdown)	Additional	IMPLEMENTED
Cross-encoder or learned reranker	—	NOT IMPLEMENTED
Semantic (trained) embeddings as the default	—	NOT THE DEFAULT — available as an alternate configuration (OpenAI/bge_m3), but the shipped default is offline hashing

35 Testing Strategy

69 automated backend tests across 7 files, run with pytest against a dedicated, freshly migrated and seeded ipsakti_test database (not the development database). No frontend unit-test suite, no browser-automation regression suite committed to the

repository, and no dedicated load-testing harness were found — frontend and live-flow verification in this project has so far been performed through manual live-browser sessions rather than a committed, repeatable test file.

FILE	TESTS	COVERS
test_auth_security.py	15	Registration/login, password rules, session expiry, CSRF, workspace isolation, full admin/user access-rules matrix
test_rag.py	16	Retrieval relevance, jurisdiction filtering/mismatch handling, citation verdicts, clarification/abstention/refusal flows, Hindi
test_retrieval.py	10	Empty-query handling, no-result handling, PDF parsing/chunking, injection flagging
test_governance.py	9	Coverage matrix honesty, provision mapping, retention, privacy, OCR, confidence heuristic disclosure
test_jurisdiction_patents.py	9	Jurisdiction-scoped patent matching, admin analytics rollups, AU gap parity, upload jurisdiction validation
test_llm_mode.py	7	Behaviour under a configured (fake) LLM provider vs. extractive mock mode
test_integration_flow.py	3	The full user→workspace→innovation→profile→research→verify→patents→regulatory→gaps→escalation→report→audit journey

All 69 tests pass as of this report (verified by executing `pytest -q` immediately before report preparation). This is a snapshot, not a continuously enforced gate — see §28, no CI pipeline runs this suite automatically on every change.

36 Evaluation Framework

An evaluation harness exists (`services/evaluation.py`, `EvaluationQuestion/EvaluationRun` tables, `POST /api/admin/evaluations/run`): a seeded set of gold questions (with expected abstention/keywords/source titles per question) is run against the live pipeline, with baselines for comparison (BM25-only, always-answer, LLM-only-without-retrieval) and classification-agreement scoring. This is a genuine evaluation mechanism, not a mock — but its outputs are whatever the last admin-triggered run produced against the current small seed corpus; there is no continuously tracked, externally validated benchmark result to report.

METRIC FAMILY	STATUS
Retrieval (Precision@K, Recall@K, MRR, NDCG)	NOT MEASURED as a tracked/published figure — the harness can compute agreement/accuracy against the gold set on a run-by-run basis, but no number is reported here since none is a stable, audited figure
Generation (faithfulness, groundedness, citation correctness/completeness)	NOT MEASURED as a published figure, though citation verification (§23) produces a real per-claim verdict at run time
System (latency, throughput, error rate)	NOT MEASURED — <code>GET /api/admin/metrics</code> , queried live for this report, returned <code>queries_sampled: 0</code> and null latency percentiles for the current window
Safety (unsupported-claim rate, abstention quality)	NOT MEASURED as a tracked figure; mechanism exists and is unit-tested (§35) but no aggregate rate is published

This section exists specifically to satisfy the report’s factuality rule. No precision, recall, F1, accuracy, or latency number appears anywhere else in this document, because none was available to verify at report time beyond the structural fact that the harness and its tables exist and run successfully.

37 Performance & Scalability

No load test, benchmark, or production traffic has been run against this deployment; the live `/api/admin/metrics` endpoint (queried for this report) reports zero sampled queries and null latency in its current window. Architecturally: retrieval and reranking run in-process inside the FastAPI worker (CPU-bound SQL queries, no external compute), embeddings default to an offline hashing scheme (no network call, effectively free but semantically weak), and background ingestion jobs run as FastAPI BackgroundTasks within the same process rather than a separate worker pool — appropriate for the current small corpus, but a self-acknowledged limitation for larger ingestion volume. Rate limiting is in-process, so correctness under multiple server instances is not guaranteed without a shared store. Status: **NOT MEASURED** (no benchmark), architecture reviewed as **PROTOTYPE-SCALE** .

38 Deployment Architecture

Local/single-machine deployment is fully scripted: Docker Compose brings up PostgreSQL (`pgvector/pgvector:pg16`); a backend Dockerfile and frontend Dockerfile exist for containerised builds; a one-command script (`scripts/start_all.sh`) additionally handles local Ollama startup/model preload, Alembic migration, seeding, and launching an ad-hoc public HTTPS tunnel (Cloudflare quick tunnel) for demonstration purposes. There is no committed Kubernetes manifest, no cloud infrastructure-as-code, no managed-database configuration, and no CI/CD pipeline of any kind. The demonstration public URL is explicitly ephemeral (tied to

one machine staying powered on) and is not a production hosting arrangement. Status: **IMPLEMENTED** for local/Docker deployment; **NOT IMPLEMENTED** for any managed/cloud production deployment path.

39 Failure Handling

FAILURE	HANDLING
LLM provider failure/timeout	Falls back to extractive-only output; states the fallback explicitly in the answer
Vector/full-text search failure	Propagates as a structured error via the standard envelope; no silent empty result is presented as "no evidence"
Source/document unavailable or rejected	Ingestion job marked FAILED with a recorded error step, visible to admins
Invalid upload (bad type, oversized, malicious PDF)	Rejected at validation, before parsing
Invalid input generally	Pydantic validation error, HTTP 422, structured error envelope
Missing citation / unverifiable claim	Sentence dropped from final answer, reported under limitations
Insufficient evidence	Explicit abstention, not a best-effort guess
Authentication failure	401/403 distinguished (no session vs. wrong scope/role), never a generic 500
Unhandled server exception	Centralised handler returns the standard error envelope; stack traces are never returned to the client

40 Complete Feature Inventory

FEATURE	CATEGORY	STATUS	EVIDENCE
User auth (register/login/logout/sessions)	Supporting	IMPLEMENTED	api/endpoints/auth.py
Separate admin console (own session/login/RBAC gate)	Additional	IMPLEMENTED	admin_console.py, lib/admin.tsx
Workspace RBAC (5 roles)	Supporting	IMPLEMENTED	models/orm.py::WorkspaceRole
Innovation profiler wizard	PS-045 Required	IMPLEMENTED	innovations/new/page.tsx
Terminology engine (ambiguity-aware)	PS-045 Required	IMPLEMENTED	services/terminology.py
Hybrid retrieval (dense+lexical+metadata+graph)	PS-045 Required	IMPLEMENTED	services/retrieval.py
Deterministic reranking	Supporting	IMPLEMENTED	retrieval.py
AI Research Assistant chat	PS-045 Required	IMPLEMENTED	api/endpoints/chat.py
Citation verification (10-point)	PS-045 Required	IMPLEMENTED	services/verification.py
Patent/prior-art matching (jurisdiction-scoped)	PS-045 Required	IMPLEMENTED	services/patents.py
Scientific evidence (ingredient vs. formulation)	PS-045 Required	IMPLEMENTED	services/analysis.py
Traditional knowledge context	PS-045 Required	PARTIAL (thin corpus)	§18
Regulatory classification (IN/US/AU)	PS-045 Required	IMPLEMENTED	services/regulatory.py
Jurisdiction engine (selection→retrieval→patents→gaps)	PS-045 Required	IMPLEMENTED	§21
Evidence gaps + risk map	PS-045 Required	IMPLEMENTED	compute_gaps(), risk_map()
Evidence graph (interactive)	Additional	IMPLEMENTED	innovations/[id]/graph
Human escalation workflow	PS-045 Required	IMPLEMENTED	services/packets.py
Evidence report generation (+Markdown export)	Additional	IMPLEMENTED	app/reports
Coverage matrix (jurisdiction×domain, computed/honest)	Additional	IMPLEMENTED	services/governance.py
Feedback queue (report wrong answer)	Additional	IMPLEMENTED	api/endpoints/review.py
Audit trail + CSV export	Supporting	IMPLEMENTED	AuditLog
Multi-provider LLM abstraction + extractive fallback	Additional	IMPLEMENTED	services/llm.py
Multilingual UI (EN/HI)	PS-045 Required	IMPLEMENTED	lib/i18n.ts
Prompt-injection detection (query + document)	Additional	IMPLEMENTED	query_understanding.py
Document ingestion + OCR	Supporting	IMPLEMENTED	services/ingestion.py
Automated test suite (69 tests)	Additional	IMPLEMENTED	§35
RAG/citation admin monitoring (incl. by-jurisdiction)	Additional	IMPLEMENTED	§30
Evaluation harness	Additional	IMPLEMENTED mechanism	§36
Trained semantic embeddings as default	Supporting	NOT DEFAULT	§8, §37
TKDL integration	—	NOT IMPLEMENTED (BY DESIGN)	§15, §18
CI/CD pipeline	—	NOT IMPLEMENTED	§8, §28
Cloud/production deployment	—	NOT IMPLEMENTED	§38
Published performance/accuracy benchmarks	—	NOT MEASURED	§36, §37
Frontend automated test suite	—	NOT IMPLEMENTED	§35

41 Additional Implementations Beyond PS-045

PS-045, as a domain problem statement, describes *what* the assistant must reason about (Ayurveda IP, TK, scientific evidence, regulation, jurisdictions, multilingual interaction, citation verification, evidence gaps, escalation, auditability). It does not itself

mandate the specific engineering apparatus below; these were added by the implementation to make the required capabilities trustworthy and operable, and are documented explicitly rather than folded silently into “PS-045 required.”

FEATURE	WHY IT EXISTS	STATUS
Fully separate Admin Console (own login/session/RBAC, re-checked every request)	A domain-correctness feature (jurisdiction-aware RAG) is only trustworthy if the corpus feeding it is curated and monitored by someone whose access is itself provably isolated from ordinary users	IMPLEMENTED
RAG & citation quality monitoring dashboards, including jurisdiction breakdowns	Without visibility into retrieval failures and citation failures, evidence-grounding claims would be unverifiable in production, not just in a demo	IMPLEMENTED
Coverage matrix (what is actually ingested, per jurisdiction × domain, Built/Partial/Planned/N/A)	Directly prevents the system from implying jurisdiction support it does not have	IMPLEMENTED
69-test automated suite covering security, retrieval, citation, jurisdiction and the full user journey	Evidence-grounding and access-isolation claims need regression protection, not just initial correctness	IMPLEMENTED
Prompt-injection detection in both queries and ingested documents	A RAG system that treats retrieved text as instructions is unsafe by construction; this is closed explicitly	IMPLEMENTED
Multi-provider LLM abstraction with automatic extractive fallback	Lets the system run (in extractive mode) with zero API cost and no external dependency, rather than requiring a paid key to function at all	IMPLEMENTED
Document review/supersession workflow (approve/reject/restore/mark superseded)	A corpus that can only grow, never be corrected, degrades trust over time	IMPLEMENTED
Feedback queue for user-reported wrong answers/citations	A closed loop for catching verification misses that automated tests don't cover	IMPLEMENTED
Evidence graph visualisation	Makes the provenance chain (feature→evidence→regulation→jurisdiction) inspectable, not just asserted in text	IMPLEMENTED

42 Complete Database Inventory

29 tables, all in one PostgreSQL schema (backend/app/models/orm.py). Full table-by-table detail:

#	TABLE	PURPOSE	KEY RELATIONSHIPS
1	users	Account/identity	Referenced by nearly every other table via created_by/user_id
2	organizations	Tenant grouping that owns workspaces	1:N workspaces
3	workspaces	Main tenancy/collaboration unit	1:N workspace_members, innovations, conversations
4	workspace_members	User↔workspace role join	unique(workspace_id,user_id)
5	innovations	The central research subject	1:1 innovation_profiles; 1:N innovation_features
6	innovation_profiles	Structured extraction of composition/use/claims	unique innovation_id
7	innovation_features	Typed, discrete feature items (drives patent matching)	FK innovation_id
8	terms	Multilingual terminology/ontology lookup	standalone
9	sources	Registry of external authorities/publishers	1:N documents
10	documents	Ingested document record	1:N document_chunks; self-FK supersedes/superseded_by
11	document_chunks	Chunk-level text + embedding + tsvector	pgvector HNSW index; GIN tsvector index
12	patents	Structured patent record (1:1 with a PATENT document)	unique document_id
13	scientific_studies	Structured study record (1:1 with a STUDY document)	unique document_id
14	regulatory_pathways	Per-jurisdiction regulatory framework/requirements	FK source_document_id
15	conversations	Chat thread, optionally tied to an innovation	1:N messages
16	messages	Single chat turn + structured answer payload	FK conversation_id
17	retrieval_results	Per-chunk retrieval trace	FK conversation_id, message_id, document_id, chunk_id
18	evidence	Passage linked to an innovation as supporting evidence	unique(innovation_id,chunk_id,evidence_type)
19	claims	Extracted claim with risk/support classification	FK message_id, innovation_id
20	claim_evidence	Claim↔chunk verification link + entailment score	FK claim_id
21	patent_feature_matches	Feature↔patent prior-art match	unique(innovation_feature_id,patent_id)

#	TABLE	PURPOSE	KEY RELATIONSHIPS
22	evidence_gaps	Missing-evidence category/risk	FK innovation_id
23	escalations	Human-review packet + status workflow	FK innovation_id, assigned_to
24	reports	Generated evidence report (JSON + Markdown)	FK innovation_id
25	ingestion_jobs	Ingestion pipeline run tracking	FK source_id, document_id
26	evaluation_questions	Gold-standard evaluation question	unique key
27	evaluation_runs	Recorded evaluation-suite run + metrics	standalone
28	feedback	User-reported issue + review workflow	FK message_id, innovation_id
29	audit_logs	Append-only action audit trail	FK user_id (indexed), action (indexed)

10 application-level enums (VARCHAR-backed, not native Postgres enum types): UserRole, WorkspaceRole, Confidentiality, InnovationStatus, FeatureType, RetrievalMethod, SupportStatus, Severity, EscalationStatus, EscalationType. 4 Alembic migrations exist.

43 Complete API Inventory

108 distinct route definitions, 112 registrations counting the health router’s dual mount. All under /api unless noted.

METHOD	PATH	PURPOSE	AUTH
auth.py (10)			
POST	/auth/register	Create account + personal workspace	None
POST	/auth/login	App sign-in	None (throttled)
POST	/auth/logout	App sign-out	User
GET	/auth/session	Session state (never 401)	None
GET	/auth/me	Current user + workspaces	User
PATCH	/auth/me	Update own profile	User
POST	/auth/change-password	Change own password	User
POST	/auth/admin/login	Admin console sign-in	None (role=ADMIN required)
POST	/auth/admin/logout	Admin console sign-out	None
GET	/auth/admin/session	Admin session state (never 401)	None
workspaces.py (5)			
GET	/workspaces	List own workspaces	User
POST	/workspaces	Create workspace	User
GET	/workspaces/{id}	Workspace detail	User (member)
PATCH	/workspaces/{id}	Update workspace settings	Workspace ADMIN
POST	/workspaces/{id}/members	Add a member	Workspace ADMIN
innovations.py (24)			
GET/POST	/innovations	List / create	User / Workspace RESEARCHER
GET/PATCH/DELETE	/innovations/{id}	Detail / update / delete	User / RESEARCHER / Workspace ADMIN
POST	/innovations/{id}/profile/generate	Generate structured profile	Workspace RESEARCHER
PATCH	/innovations/{id}/profile	Edit/confirm profile	Workspace RESEARCHER
POST	/innovations/{id}/analyze	Run evidence/patent/classification/gap analysis	Workspace RESEARCHER
GET/POST	/innovations/{id}/evidence	List / add evidence	User / RESEARCHER
GET	/innovations/{id}/scientific	Scientific evidence cards	User
GET	/innovations/{id}/tk	TK context	User
GET	/innovations/{id}/patents	Patent matrix + concepts	User
PATCH	/innovations/{id}/patents/matches/{mid}	Review a patent match	Workspace REVIEWER

METHOD	PATH	PURPOSE	AUTH
GET/PATCH	/innovations/{id}/evidence-gaps/{gid}	List / update gap	User / RESEARCHER
GET	/innovations/{id}/risk-map	Risk map	User
GET	/innovations/{id}/graph	Evidence graph	User
GET/POST	/innovations/{id}/classification	Get / run classification	User / RESEARCHER
GET	/innovations/{id}/regulatory	Passport + comparison	User
GET	/innovations/{id}/workspace	Owning workspace (hidden from schema)	User
GET	/innovations/{id}/summary	Structured summary	User
GET	/innovations/{id}/provisions	Provision relevance flags	User
chat.py (6)			
POST	/chat	Ask AI Research Assistant	User (rate-limited)
GET	/conversations/{id}/messages	List/get conversation(s)/messages	User (own)
DELETE	/conversations/{id}	Delete conversation	User (own)
PATCH	/messages/{id}	Set feedback/flag	User (own)
search.py (8)			
POST	/search/{patents}/{scientific}/{regulatory}	Domain-scoped search	User (rate-limited)
POST	/citations/verify	Verify a claim against a passage	User
GET	/classification/questions	Classification question set	User
POST	/classification/analyze	Provisional classification	User (RESEARCHER if tied to an innovation)
GET	/regulatory/{jurisdiction}	Regulatory overview for IN/US/AU	User
review.py (12)			
GET/POST	/escalations/{id}	List/create/get/update escalation	User / RESEARCHER / REVIEWER
GET/POST	/reports/{id}/markdown	List/create/get report	User / RESEARCHER
GET	/audit	Workspace audit trail	User
GET/POST/PATCH	/feedback/{id}	List/create/review feedback	User / REVIEWER
corpus.py (9)			
GET	/sources	Source registry	User
GET/POST	/documents/{id}	List/upload/get documents	User / RESEARCHER
GET	/uploads/notice	Privacy notice before upload	User
GET	/ingestion-jobs/{id}	Ingestion job status	User
GET	/coverage	Jurisdiction×domain coverage matrix	User
GET	/privacy	Data handling policy	User
dashboard.py (1)			
GET	/dashboard	Workspace dashboard metrics	User
admin_console.py (28) — every route additionally requires an admin session			
GET	/admin/overview, /admin/metrics, /admin/settings	System dashboard, metrics, effective config	Admin
POST	/admin/retention/run	Apply retention policy	Admin
GET/PATCH	/admin/users/{id}	List / update users	Admin
GET	/admin/workspaces	List all workspaces	Admin
GET/POST/PATCH	/admin/sources/{id}	List/register/edit sources	Admin
GET	/admin/documents/{id}	Cross-workspace document view	Admin
POST	/admin/documents/ingest	Ingest an official document	Admin
PATCH	/admin/documents/{id}/review	Review/reject/restore/supersede	Admin

METHOD	PATH	PURPOSE	AUTH
POST	/admin/documents/{id}/reindex	Re-embed document chunks	Admin
GET	/admin/ingestion-jobs, /admin/update-queue	Job list, stale-document queue	Admin
GET	/admin/rag, /admin/citations	RAG & citation monitoring	Admin
GET/POST	/admin/escalations/{id}	List/get/assign escalations	Admin
GET/PATCH	/admin/feedback/{id}	List/review feedback	Admin
GET	/admin/audit-logs(/export)	Query / export audit log	Admin
GET/POST	/admin/evaluations(/run)	View / run evaluation suite	Admin
health.py (4, mounted at both /health and /api/health)			
GET	/health(/database)(/vector)(/llm)	Liveness/readiness checks	None

44 Complete Frontend Inventory

55 distinct routes (see the page-count table in §9 for the breakdown by area). Shared component library: 23 primitives in components/ui.tsx (Button, Card, Badge family, Spinner, ProgressSteps, EmptyState, ErrorState, PageHeader, Stat, Tabs, Drawer, Field, Notice, Popover, ProfileMenu, date formatters), plus evidence.tsx (8 domain cards), auth-form.tsx (shared login/register/admin-login form), innovation-card.tsx, back.tsx (navigation-history back button), markdown.tsx. State/lib layer: lib/api.ts (fetch wrapper + session/workspace/jurisdiction persistence), lib/providers.tsx (app-wide context), lib/admin.tsx (separate admin-session context), lib/i18n.ts (EN/HI dictionaries), lib/types.ts, lib/hooks.ts. No Redux/Zustand/Zod — React Query + Context + localStorage only.

45 Complete AI Inventory

COMPONENT	TECHNOLOGY	STATUS
Generation model (configurable)	Ollama / Gemini / Groq / Grok / OpenAI / DeepSeek / Mock	PARTIAL — provider-dependent at deploy time
Embedding model (configurable)	Hashing n-gram (default) / OpenAI text-embedding-3-small / bge_m3	PARTIAL — default is not semantic
Reranker	Deterministic weighted scoring (relevance/lexical/jurisdiction/authority/freshness/doc-type/feature-match)	IMPLEMENTED
Claim extraction	Rule-based segmentation + LLM entailment (capped 3/answer, if configured)	IMPLEMENTED
Citation verification	10-point deterministic check (§23)	IMPLEMENTED
Terminology/query expansion engine	Rule + lookup table (Term table)	IMPLEMENTED
Jurisdiction/intent/language detector	Rule-based (query_understanding.py)	IMPLEMENTED
Evaluation harness	Gold-question set + baselines (BM25-only, always-answer, no-retrieval)	IMPLEMENTED mechanism
Agents / tool-calling / multi-agent orchestration	—	NOT IMPLEMENTED

46 Original PS-045 vs. Current Implementation

PS-045 REQUIREMENT	ACTUAL IMPLEMENTATION	STATUS	GAP / EXTENSION
Evidence-grounded IP/regulatory assistance for Ayurveda	Full RAG pipeline + citation verification over a curated corpus	IMPLEMENTED	Corpus is a small seed set (§15), not a production knowledge base
Traditional knowledge context	TK-domain retrieval, access-level gating, NBA gap trigger	PARTIAL	No TKDL access by design; thin corpus
Patent / prior-art intelligence	Feature-to-patent matching, jurisdiction-scoped, family grouping	IMPLEMENTED	Only 7 fictional seed patents; never presents a patentability verdict
Scientific evidence	Study cards, ingredient-vs-formulation separation enforced in code	IMPLEMENTED	Only 6 seed studies, all ingredient-level
Jurisdiction-specific regulatory pathways (IN/US/AU)	Per-jurisdiction classification, passport, comparison, gap triggers for all three	IMPLEMENTED	None beyond corpus depth
Multilingual (EN/HI)	Full UI translation + terminology-based Hindi retrieval	IMPLEMENTED	LLM-based Hindi generation/translation is provider-dependent

PS-045 REQUIREMENT	ACTUAL IMPLEMENTATION	STATUS	GAP / EXTENSION
Citation verification	10-point deterministic check, answer rebuilt from verified sentences only	IMPLEMENTED	None material
Evidence gaps	Computed gaps with severity, incl. parity triggers for all three jurisdictions	IMPLEMENTED	None material
Human escalation	4 escalation types, full evidence packet, reviewer assignment	IMPLEMENTED	None material
Auditability	Append-only audit log, CSV export, hashed IPs	IMPLEMENTED	None material
(not explicit in PS-045) Separate admin console + RBAC	Fully isolated session/RBAC, 28 admin routes	IMPLEMENTED	Extension — see §41
(not explicit in PS-045) Automated test coverage	69 tests across security/RAG/jurisdiction/governance/integration	IMPLEMENTED	Extension — see §41

47 What Has Actually Been Built

If the current repository is run today, what does the software actually provide? A working, locally-deployable full-stack web application: a user can register, select a jurisdiction, describe an Ayurveda innovation in free text, receive a structured profile with flagged ambiguities, ask the AI Research Assistant jurisdiction-aware questions and receive citation-verified answers (or an honest abstention), see scientific evidence correctly separated by ingredient vs. formulation level, see a jurisdiction-scoped patent/prior-art matrix, get a provisional regulatory classification and passport across India/USA/Australia, see computed evidence gaps and a risk map, inspect an interactive evidence graph, generate an exportable evidence report, and raise a human escalation with a full packet. In parallel, an administrator — through a fully separate, server-enforced console — can curate the source/document corpus, monitor retrieval and citation quality (including per-jurisdiction), run the evaluation harness, manage users and workspaces, triage escalations and feedback, and audit every action. All of this is backed by a real Postgres database with 29 tables, 108 API routes, and is exercised by 69 passing automated tests. What it does *not* provide, as of this report: a production-scale verified corpus, published accuracy/latency benchmarks, a CI/CD pipeline, or a cloud deployment — these are stated plainly in §48.

48 Limitations

- Corpus is a curated seed set: 39 documents, 13 sources, 81 chunks (30 real/9 demo-fictional) — not a production knowledge base
- Patent coverage is thin and uneven across jurisdictions (3 IN / 2 US / 1 AU / 1 EP), all fictional
- Scientific evidence is ingredient-level only (6 studies); no finished-formulation-level study exists in the corpus
- No TKDL or other restricted traditional-knowledge database access, by design
- Only 3 jurisdictions supported (India, USA, Australia); all others explicitly refused, not approximated
- Only 2 languages supported (English, Hindi); Hindi retrieval relies on terminology expansion, not a native Hindi corpus
- Default embeddings are offline hashing, not a trained semantic model (semantic embeddings are available but not the default)
- LLM generation is provider-dependent; with no provider configured, answers are extractive quotes, not synthesis
- No published precision/recall/latency benchmark; queries_sampled: 0 in the live metrics endpoint at report time
- Rate limiting is in-process, not safe across multiple server instances
- Background jobs run inline in the request process, not on a dedicated worker queue
- No CI/CD pipeline; the 69-test suite is not automatically enforced on every change
- No frontend automated test suite or committed browser-regression suite
- No cloud/production deployment path (Docker Compose + a local-machine demo tunnel only)
- No third-party security audit or compliance certification held or claimed

49 Future Roadmap

The items below are proposals for future work. None are implemented today; they are listed separately from §40–§47 specifically so they are never mistaken for current capability.

- Expand the verified document corpus, especially patent and formulation-level scientific coverage
- Add jurisdictions beyond India/USA/Australia with the same hard-isolation discipline already proven for the current three
- Move the default embedding provider to a trained semantic model and reseed
- Add a cross-encoder or learned reranking stage
- Integrate additional verified source feeds (patent office APIs, regulator publication feeds)
- Build a repeatable, published evaluation benchmark (retrieval + generation + safety metrics) run on a schedule, not only on demand

- Stand up a CI/CD pipeline running the existing 69-test suite (and a new frontend suite) on every change
 - Move rate limiting and background ingestion to shared infrastructure (e.g. Redis, a worker queue) for multi-instance deployment
- Add a cloud deployment target with managed Postgres/pgvector and observability
 - Commission an independent security review before handling real confidential innovation data at scale

50 Technical Differentiation

VS.	WHAT THIS SYSTEM DOES DIFFERENTLY (AS IMPLEMENTED, NOT ASPIRATIONAL)
Generic LLM chatbot	Answers are rebuilt from only the sentences that pass citation verification; an unsupported sentence is dropped, not shown with a caveat
Generic RAG	Retrieval filters are hard SQL constraints tied to jurisdiction, access level and review status — not just a similarity threshold
Generic patent search	Matching is scoped to the innovation's own declared target markets and is explicitly never presented as a patentability or infringement determination
Generic regulatory portal	Classification runs materially different rule logic per jurisdiction (not a templated label swap) and always discloses its own assumptions and open questions
Any of the above	A fully separate, re-verified-per-request admin session; a computed (not asserted) coverage matrix; a 69-test regression suite covering the access-isolation and jurisdiction-correctness claims made in this report

A Technical Status Legend

IMPLEMENTED	Fully functional and verified — the complete chain (frontend → API → backend → database/service → AI/RAG → response) works, and is exercised by a passing test or a direct live check performed for this report.	PARTIAL	Functionality exists but completeness or integration is limited — e.g. provider-dependent, or backed by a thin dataset.
PROTOTYPE	Demonstration-level implementation, not validated at scale.	MOCK	Uses dummy/static/fictional data or behaviour (clearly labelled as such in the product itself).
PLANNED	Documented or intended, but no functional implementation exists.	NOT IMPLEMENTED	No functional implementation found during inspection.
NOT MEASURED	The capability exists, but no quantitative evaluation of it is available.		

B Repository Evidence Index

All file paths referenced in this report are relative to the repository root. Primary directories inspected: backend/app/models/orm.py, backend/app/schemas.py, backend/app/api/endpoints/*.py (9 files), backend/app/services/*.py (research, retrieval, verification, terminology, profile, patents, regulatory, analysis, governance, ingestion, evaluation, llm, audit, query_understanding, packets), backend/app/seed/corpus.py and run.py, backend/alembic/, frontend/app/** (55 routes), frontend/components/*.tsx, frontend/lib/*.ts(x), tests/*.py (7 files, 69 tests), docker-compose.yml, docker/*.Dockerfile, scripts/start_all.sh, .env.example, README.md. Live endpoints queried directly to source figures for this report: GET /health, GET /api/coverage, GET /api/admin/metrics. This index exists so every factual claim above can be independently re-verified against the repository rather than taken on trust.

End of report. IP-SAKTI Sahayak — Technical Architecture & Implementation Report, v1.0, Problem Statement 045.